TrueSaya Work Items with Topics List = Target Next Venture

content

- Microservices Architecture: Design Patterns & Principles
  - Enterprise E-Commerce Example
    - Reference architecture used throughout
- 1. Decomposition Patterns
  - 1.1 Decompose by Business Capability
  - 1.2 Decompose by Subdomain / DDD Bounded Context
  - 1.3 Single Responsibility Service
  - 1.4 Strangler Fig Pattern
  - 1.5 Modular Monolith First
  - 1.6 Self-Contained Services
  - 1.7 Domain-Driven Design
- 2. Integration & Communication Patterns
  - Synchronous Communication
    - 2.1 API Gateway
    - 2.2 Backend for Frontend (BFF)
    - 2.3 API Composition
    - 2.4 Aggregator Pattern
    - 2.5 Service-to-Service REST
    - 2.6 GraphQL Federation
- 3. Asynchronous Communication
  - 3.1 Event-Driven Architecture
  - 3.2 Publish-Subscribe
  - 3.3 Event Notification
  - 3.4 Event-Carried State Transfer
  - 3.5 Domain Events
  - 3.6 Message Broker
- 4. Service Discovery Patterns
  - Core problem
  - Client-Side Discovery
  - Server-Side Discovery
  - Service Registry
  - DNS-Based Discovery
- 5. Data Management Patterns
  - 5.1 Database per Service
  - 5.2 Shared Database
  - 5.3 Saga
  - 5.4 Transactional Outbox
  - 5.5 CQRS
  - 5.6 Event Sourcing
  - 5.7 Change Data Capture
  - 5.8 Materialized View
  - 5.9 Polyglot Persistence
  - 5.10 Cache-Aside

- 5.11 Read Replica
- 6. Resilience & Fault-Tolerance
  - Circuit Breaker
  - Retry
  - Timeout
  - Bulkhead
  - Fallback
  - Rate Limiting
  - Throttling
  - Dead Letter Queue
  - Health Check
  - Leader Election
  - Graceful Degradation
- 7. Distributed Transaction Patterns
  - Saga Orchestration
  - Saga Choreography
  - Two-Phase Commit
  - Compensating Transaction
  - TCC
- 8. Deployment Patterns
- 9. Observability Patterns
  - Centralized Logging
  - Distributed Tracing
  - Correlation ID
  - Metrics
  - Audit Logging
  - OpenTelemetry
- 10. Security Patterns
  - Typical security architecture
  - OAuth2
  - JWT
  - Zero Trust
  - mTLS
  - Service Identity
  - Secret Management
  - Token Propagation
- 11. Infrastructure Patterns
  - Sidecar
  - Ambassador
  - Adapter
  - Service Mesh
  - Anti-Corruption Layer
  - External Configuration
- 12. Cloud-Native & Kubernetes Patterns
  - Operator Pattern
  - GitOps
  - Workload Identity
  - Multi-Cluster Architecture

- Cell-Based Architecture
- Platform Engineering / IDP

13. Workflow & Orchestration Patterns
    - Orchestration
    - Choreography
    - Workflow Engine
    - Process Manager
    - Long-Running Transaction
14. Caching Patterns
15. Emerging / Modern Distributed-System Patterns
    - Event Streaming Architecture
    - AI Gateway
    - Data Mesh Integration
    - AIOps
    - Edge AI Microservices
    - Agent-Based Services

Enterprise Principles Behind All These Patterns
- High Cohesion + Loose Coupling
- Independent Deployability
- Design for Failure

Most Important Patterns for Your Java/Spring Boot Interview
- Tier 1: Must Know
- Tier 2: Senior Developer / Lead
- Tier 3: Architect

One Complete E-Commerce Order Flow

1. Concise Pattern-to-Pattern Comparison

3. Common Interview Questions with Concise Answers
    1. What is a microservice?
    2. How do you identify microservice boundaries?
    3. Why use microservices instead of a monolith?
    4. What is Database per Service?
    5. Why should services not share database tables?
    6. When should REST communication be used?
    7. When should asynchronous messaging be used?
    8. API Gateway vs BFF?
    9. What is a Saga?
    10. Saga orchestration vs choreography?
    11. What is a compensating transaction?
    12. What is the Transactional Outbox Pattern?
    13. What problem does CQRS solve?
    14. CQRS vs Event Sourcing?
    15. How do you handle duplicate messages?
    16. What is eventual consistency?
    17. How do you make REST calls resilient?
    18. When should a request be retried?
    19. What is a Circuit Breaker?
    20. What is the Bulkhead Pattern?
    21. How does service discovery work in Kubernetes?

22. Is Eureka required on Kubernetes?
23. How do you secure Spring Boot microservices?
24. How do you implement observability?
25. Liveness vs readiness probe?
26. How do you cache product data?
27. How do you deploy without downtime?
28. How do you migrate a monolith to microservices?
29. What are the main microservice challenges?
30. Explain the complete order flow.

A **concise, interview-oriented and enterprise-project-oriented revision sheet**, specifically mapped to an **E-Commerce Platform using Java, Spring Boot Microservices, Spring, Hibernate/JPA, REST APIs, AWS, Docker, and Kubernetes**.

Microservices Architecture: Design Patterns & Principles

It Organized the subject into **14 major categories**, ranging from decomposition and communication through data, resilience, deployment, observability, security, Kubernetes/cloud-native, workflow, caching, and emerging distributed-system patterns.

Enterprise E-Commerce Example

Reference architecture used throughout

Assume the e-commerce platform contains:

```
Web / Mobile Clients
        |
   API Gateway
        |
+-------+---------+---------+---------+
|                 |         |         |
Catalog         Cart      Order    Customer
Service        Service    Service   Service
                           |
              +------------+-------------+
              |                          |
          Payment                    Inventory
          Service                    Service
              |
        Notification
          Service

Async Events -> Kafka / AWS SNS/SQS
Databases    -> PostgreSQL / MySQL / DynamoDB
Cache        -> Redis
Runtime      -> Docker + Kubernetes / AWS EKS
Monitoring   -> OpenTelemetry / CloudWatch

```

A typical implementation would use:

Java + Spring Boot + Spring Web + Spring Data JPA + Hibernate + REST APIs + Kafka/SQS + Redis + Docker + Kubernetes/EKS + AWS.

---

1. Decomposition Patterns

The document identifies Business Capability, DDD/Subdomain, Single Responsibility, Strangler Fig, Modular Monolith First, Self-Contained Services and DDD as decomposition approaches. [List od Mi...iples.docx | Word]

1.1 Decompose by Business Capability

Idea: Split the application according to what the business does.

E-Commerce:

```
Catalog
Order
Payment
Inventory
Shipping
Customer
Notification

```

Each becomes an independently deployable Spring Boot service.

Key principle: Services should follow **business boundaries**, not technical layers.

Bad:

```
Controller-Service
Database-Service
Validation-Service

```

Better:

```
Order-Service
Payment-Service
Inventory-Service

```

1.2 Decompose by Subdomain / DDD Bounded Context

Use **Domain-Driven Design** to discover independent business domains.

Example:

```
Ordering Context
 └─ Order, OrderItem

Payment Context
 └─ Payment, Refund

Inventory Context
 └─ Stock, Reservation

Shipping Context
 └─ Shipment, Tracking

```

An `Order` in Ordering does not have to use the same model as an `Order` concept used by Shipping.

Principle: Each bounded context owns its **domain model and business terminology**.

1.3 Single Responsibility Service

A microservice should have **one clear business responsibility and one main reason to change**.

Example:

```
Payment Service
 -> authorize payment
 -> capture payment
 -> refund payment

```

Do not put inventory, shipment and customer-profile logic inside Payment Service.

1.4 Strangler Fig Pattern

Used for **gradual monolith-to-microservices migration**.

```
Client
   |
Gateway
 /     \
Legacy   New Spring Boot
Monolith Microservices

```

Example:

1. Existing e-commerce monolith contains Catalog.
2. Build new `catalog-service`.
3. Gateway redirects `/products/**` to the new service.
4. Remove Catalog functionality from monolith.
5. Repeat for Orders, Payments, etc.

Use when: Replacing a large legacy Java/J2EE/Spring monolith incrementally.

1.5 Modular Monolith First

Build one application but maintain **strict internal module boundaries**:

```
ecommerce
 ├─ catalog
 ├─ ordering
 ├─ payment
 └─ inventory

```

Extract modules into microservices only when independent scaling/deployment is justified.

Benefit: Avoids premature distributed-system complexity.

1.6 Self-Contained Services

Each service should ideally own:

- Business logic
- API
- Data
- Deployment
- Runtime configuration

Example:

```
Order Service
├─ REST API
├─ Domain Logic
├─ Spring Data Repository
└─ Order DB

```

1.7 Domain-Driven Design

DDD aligns software services with business domains.

Typical building blocks:

```
Entity
Value Object
Aggregate
Aggregate Root
Repository
Domain Service
Domain Event
Bounded Context

```

Example:

```
Order
 ├─ OrderId
 ├─ CustomerId
 ├─ List<OrderItem>
 └─ OrderStatus

```

`Order` can act as the **aggregate root** controlling order consistency.

---

2. Integration & Communication Patterns

The document separates microservice communication into **synchronous and asynchronous integration patterns**. [List od Mi...iples.docx | Word]

Synchronous Communication

2.1 API Gateway

Single controlled entry point to backend services.

```
Client
  |
API Gateway
  |
  +-- Catalog Service
  +-- Order Service
  +-- Customer Service
  +-- Payment Service

```

Responsibilities:

- Routing
- Authentication
- Rate limiting
- TLS termination
- Logging
- Request filtering

Possible implementation:

AWS API Gateway / Spring Cloud Gateway.

2.2 Backend for Frontend (BFF)

Create different backend APIs for different client experiences.

```
Mobile -> Mobile BFF ----\
                          -> Microservices
Web ----> Web BFF -------/

```

Example:

Mobile requires:

```
Product
Price
Thumbnail
Stock

```

Desktop may require:

```
Product
Price
Large Images
Recommendations
Reviews
Stock
Related Products

```

Benefit: Prevents one generic API from becoming overly complicated.

2.3 API Composition

A service/API builds one response by calling multiple microservices.

Example:

```
GET /order-summary/1001

Order Service ------\
Payment Service ----- > Composition -> Response
Shipping Service ---/

```

Useful when UI requires information owned by multiple bounded contexts.

2.4 Aggregator Pattern

An aggregator coordinates multiple service calls and returns a unified result.

Example:

```
ProductPageAggregator
   |
   +-> Catalog
   +-> Inventory
   +-> Pricing
   +-> Reviews

```

Difference from basic API composition: Aggregator usually represents an explicit orchestration component responsible for the combined business/UI response.

2.5 Service-to-Service REST

Spring Boot services communicate using HTTP.

Example:

```
Order Service
     |
 GET /inventory/{sku}
     |
Inventory Service

```

Modern Spring:

```
@HttpExchange("/inventory")
public interface InventoryClient {

    @GetExchange("/{sku}")
    InventoryResponse find(@PathVariable String sku);
}

```

Use synchronous REST when the caller **requires an immediate response**.

Avoid building deep chains such as:

```
A -> B -> C -> D -> E

```

because latency and failure probability increase.

2.6 GraphQL Federation

Several domains expose GraphQL schemas that are combined into a unified graph.

Useful for complex storefront queries requiring:

```
Product
 + Inventory
 + Price
 + Reviews
 + Recommendations

```

Particularly useful at the **experience/API layer**, rather than replacing all internal microservice communication.

---

3. Asynchronous Communication

The document includes Event-Driven Architecture, Pub/Sub, Event Notification, Event-Carried State Transfer, Event Sourcing, CQRS, Domain Events and Message Broker. [List od Mi...iples.docx | Word]

3.1 Event-Driven Architecture

Services communicate by publishing events rather than directly calling each other.

```
Order Service
     |
 OrderCreated
     |
 Kafka / SNS
   /   |    \
Inventory Payment Notification

```

Excellent for decoupled e-commerce workflows.

3.2 Publish-Subscribe

One publisher sends an event that can be consumed independently by multiple subscribers.

```
OrderPlaced
   |
 Broker
 /  |   \
Payment
Inventory
Analytics

```

Publisher does not need to know every consumer.

3.3 Event Notification

Publish a small event informing consumers that something happened:

```
{
  "eventType": "ProductPriceChanged",
  "productId": "P100"
}

```

Consumer obtains additional data if required.

Advantage: Small events.

Trade-off: May trigger additional service/API calls.

3.4 Event-Carried State Transfer

Include the useful changed state inside the event.

```
{
  "eventType": "ProductPriceChanged",
  "productId": "P100",
  "price": 1299.00,
  "currency": "INR"
}

```

Consumers can update their own local read models without calling Product Service.

Trade-off: Larger payload and event schema-management requirements.

3.5 Domain Events

Represent important things that have already happened in the business.

Examples:

```
OrderPlaced
PaymentAuthorized
InventoryReserved
OrderShipped
PaymentFailed
OrderCancelled

```

Use **past-tense business names** because events represent facts.

3.6 Message Broker

Broker decouples producers and consumers.

Examples:

```
Apache Kafka
AWS SNS/SQS
RabbitMQ

```

Producer:

```
kafkaTemplate.send("orders", orderCreatedEvent);

```

Consumers independently process the event.

---

4. Service Discovery Patterns

The document calls out Client-Side Discovery, Server-Side Discovery, Service Registry and DNS-Based Discovery, with Eureka, Consul and Kubernetes as examples. [List od Mi...iples.docx | Word]

Core problem

Pods/containers are dynamic.

Do not hardcode:

```
http://10.1.2.45:8080

```

Services need a stable method to locate one another.

Client-Side Discovery

```
Service A
   |
Registry Lookup
   |
Select Instance
   |
Service B

```

Client chooses an instance.

Server-Side Discovery

```
Service A
   |
Load Balancer
   |
Registry
   |
Service B instances

```

Infrastructure chooses the target.

Service Registry

Maintains available service instances.

Traditional example:

```
Spring Cloud Netflix Eureka

```

DNS-Based Discovery

For your Kubernetes-based environment, this is particularly important:

```
order-service
inventory-service
payment-service

```

Kubernetes Services provide stable DNS identities while backing Pods can change.

Practical choice: Prefer **native Kubernetes service discovery** when applications already run on Kubernetes rather than adding Eureka unnecessarily.

---

5. Data Management Patterns

The document includes Database per Service, Shared Database, Saga, Transactional Outbox, Event Sourcing, CQRS, CDC, Materialized View, Polyglot Persistence, Cache-Aside and Read Replica. [List od Mi...iples.docx | Word]

5.1 Database per Service

Each microservice owns its data.

```
Order Service ------ Order DB
Payment Service ---- Payment DB
Inventory Service -- Inventory DB

```

Payment Service must **not directly query Order Service's tables**.

Access data via:

```
API
or
Events

```

This is one of the most important microservice principles.

5.2 Shared Database

Multiple services access one database.

```
Order ----\
Payment --- > Shared Ecommerce DB
Inventory -/

```

Generally an **anti-pattern for mature microservices**, but practical as a transitional strategy during legacy migration.

Risks:

- Tight coupling
- Schema conflicts
- Deployment dependencies
- Weak service ownership

5.3 Saga

Handles transactions spanning multiple services without one global ACID transaction.

Example:

```
Place Order
   ↓
Reserve Inventory
   ↓
Authorize Payment
   ↓
Create Shipment

```

Failure:

```
Payment fails
   ↓
Release Inventory
   ↓
Cancel Order

```

Uses **local transactions + compensating actions**.

5.4 Transactional Outbox

Solves the dual-write problem.

Bad:

```
save order DB
publish Kafka event

```

Database commit could succeed while event publication fails.

Better:

```
Single DB Transaction
      |
+-----+------+
|            |
Order      Outbox
Table       Table
              |
             CDC
              |
            Kafka

```

Spring/JPA:

```
@Transactional
public void createOrder(Order order) {
    orderRepository.save(order);
    outboxRepository.save(
        OutboxEvent.orderCreated(order)
    );
}

```

Both records commit atomically.

5.5 CQRS

Command Query Responsibility Segregation

Separate:

```
WRITE model -> Commands
READ model  -> Queries

```

Example:

```
POST /orders
      |
Order Write Model
      |
OrderPlaced Event
      |
Read Model
      |
GET /order-history

```

Useful where e-commerce write and read workloads differ significantly.

5.6 Event Sourcing

Instead of storing only current state, store domain events.

Traditional:

```
Order
Status = SHIPPED

```

Event sourced:

```
OrderCreated
PaymentAuthorized
InventoryReserved
OrderPacked
OrderShipped

```

Current state is reconstructed from events.

Important: Powerful but significantly more complex. Do not use for every CRUD microservice.

5.7 Change Data Capture

Capture committed database changes and publish them downstream.

Typical architecture:

```
PostgreSQL/MySQL
```

```
     |
Transaction Log
     |
Debezium
     |
Kafka

```

Especially valuable with the **Outbox Pattern**.

5.8 Materialized View

Create a precomputed view optimized for reads.

Example Product Search view:

```
Product
Price
Availability
Rating
Category

```

Data may originate from several services.

Very useful with **CQRS**.

5.9 Polyglot Persistence

Choose databases based on workload.

Example enterprise e-commerce architecture:

```
Orders      -> PostgreSQL
Catalog     -> DynamoDB
Search      -> OpenSearch
Cart        -> Redis
Events      -> Kafka
Documents   -> S3

```

Principle: Database technology follows use case, not organizational standardization alone.

5.10 Cache-Aside

Application manages cache explicitly.

```
GET Product
   |
Redis hit? --Yes--> Return
   |
   No
   ↓
Database
   |
Store Redis
   |
Return

```

Ideal for frequently read Catalog/Product information.

5.11 Read Replica

```
Writes -> Primary DB
             |
         Replication
             |
Reads --> Replica(s)

```

Useful for high-volume order-history/reporting queries.

Remember that replicas may introduce **replication lag/eventual consistency**.

---

6. Resilience & Fault-Tolerance

The document lists Circuit Breaker, Retry, Timeout, Bulkhead, Fallback, Rate Limiting, Throttling, DLQ, Health Check, Leader Election and Graceful Degradation. [List od Mi...iples.docx | Word]

Circuit Breaker

Stop repeatedly calling a failing dependency.

```
CLOSED
  ↓ failures
OPEN
  ↓ wait
HALF-OPEN
  ↓ success
CLOSED

```

Spring Boot:

```
@CircuitBreaker(name = "inventory", fallbackMethod = "fallback")
public Inventory getInventory(String sku) {
    return inventoryClient.get(sku);
}

```

Common library: **Resilience4j**.

Retry

Retry transient failures.

```
Request
  ↓
Fail
  ↓
Wait / backoff
  ↓
Retry

```

Use for transient network problems.

Never blindly retry non-idempotent operations, particularly payment capture.

Timeout

Never allow dependent calls to wait indefinitely.

```
Order -> Payment
        maximum permitted duration

```

Timeouts should be explicitly configured throughout the call chain.

Bulkhead

Isolate resources to prevent one failure from exhausting the whole application.

```
Payment Thread Pool
Inventory Thread Pool
Recommendation Thread Pool

```

If Recommendations fail under load, checkout should remain available.

Fallback

Return a degraded alternative.

Example:

```
Recommendation service unavailable
       ↓
Show "Popular Products"

```

Do not use fake success as a fallback for critical payment/inventory operations.

Rate Limiting

Control requests per consumer/time window.

Example:

```
100 requests/sec/client

```

Useful at API Gateway for login, product lookup and checkout APIs.

Throttling

Reduce/shape consumption when limits or resource capacity are reached.

Rate limiting protects APIs; throttling additionally manages **traffic/resource consumption**.

Dead Letter Queue

Messages that repeatedly fail processing move to:

```
Main Queue
   ↓ retries exhausted
DLQ

```

Operations can inspect, correct and replay them.

Critical for Order/Payment event processing.

Health Check

Spring Boot Actuator:

```
/actuator/health

```

Kubernetes:

```
Liveness Probe
Readiness Probe
Startup Probe

```

Readiness asks: **Should this instance receive traffic?**

Liveness asks: **Should Kubernetes restart it?**

Leader Election

Exactly one replica performs a specific coordination task.

Example:

```
5 Scheduler Pods
    |
1 elected leader
    |
Run reconciliation job

```

Graceful Degradation

Keep core commerce features working even when non-critical services fail.

Example:

```
Recommendations down -> Checkout still works.
Reviews down          -> Product ordering still works.

```

---

7. Distributed Transaction Patterns

The document specifically identifies Saga Orchestration, Saga Choreography, 2PC, Compensation and TCC. [List od Mi...iples.docx | Word]

Saga Orchestration

Central coordinator:

```
Order Saga
 |
 +-> Inventory
 |
 +-> Payment
 |
 +-> Shipping

```

Coordinator explicitly decides the next step.

Good for: Complicated checkout workflows requiring clear visibility.

Saga Choreography

No central coordinator:

```
OrderCreated
   ↓
InventoryReserved
   ↓
PaymentAuthorized
   ↓
ShipmentCreated

```

Services react to events.

Good: Highly decoupled.

Risk: Complex event flows become difficult to understand.

Two-Phase Commit

```
Prepare
  ↓
Commit

```

Maintains atomic distributed transactions but introduces strong coordination and availability/scaling costs.

Usually avoided for loosely coupled cloud microservices.

Compensating Transaction

Undo a previously completed business operation.

```
Payment completed
Shipping creation fails
       ↓
Refund Payment
Release Inventory
Cancel Order

```

TCC

```
TRY
reserve resources

CONFIRM
commit operation

CANCEL
release resources

```

Useful when business resources can explicitly support reservations.

---

8. Deployment Patterns

The document lists Blue-Green, Canary, Rolling, Recreate, A/B, Shadow, Feature Toggle and Immutable Deployment. [List od Mi...iples.docx | Word]

Pattern	Point-to-Point Meaning
Blue-Green	Run old and new environments, then switch traffic
Canary	Release new version to a small traffic percentage first
Rolling	Kubernetes gradually replaces old Pods
Recreate	Stop old version and start new one
A/B	Route groups to variants for experimentation
Shadow	Mirror production traffic to new version without affecting users
Feature Toggle	Enable/disable functionality through configuration
Immutable	Deploy new container/image rather than modifying running servers

For Kubernetes Spring Boot services, **Rolling + Canary + Feature Flags** is a strong default combination.

---

9. Observability Patterns

The document includes Centralized Logging, Distributed Tracing, Correlation IDs, Metrics, Health Endpoints, Audit Logging and OpenTelemetry. [List od Mi...iples.docx | Word]

Centralized Logging

```
Order logs ------\
Payment logs ----- > Central Logging
Inventory logs --/

```

On AWS:

```
CloudWatch Logs

```

Distributed Tracing

Trace one checkout across services.

```
Browser
  ↓
Gateway
  ↓
Order
  ↓
Inventory
  ↓
Payment

```

Each request carries trace/span information.

Correlation ID

```
X-Correlation-ID: ORD-719291

```

Propagate the identifier across APIs, messages and logs.

Metrics

Monitor:

```
Request rate
Error rate
Latency
CPU/Memory
JVM Heap
GC
DB connection pool
Kafka consumer lag

```

Audit Logging

Record business/security-sensitive operations:

```
Who?
What?
When?
Which resource?
Outcome?

```

Especially important for administrative, payment and refund operations.

OpenTelemetry

Standard approach for:

```
Traces + Metrics + Logs

```

Instrument Spring Boot services and export telemetry into the enterprise observability platform.

---

10. Security Patterns

The document identifies API Gateway Security, OAuth2, JWT, Zero Trust, mTLS, Service Identity, Secret Management and Token Propagation. [List od Mi...iples.docx | Word]

Typical security architecture

```
User
 ↓ OAuth2/OIDC
Identity Provider
 ↓ JWT
API Gateway
 ↓
Spring Boot Services
 ↓ workload identity / mTLS
Internal Services

```

OAuth2

Provides delegated authorization.

JWT

Carries authenticated identity/claims.

```
Header.Payload.Signature

```

Spring Security Resource Server validates JWT before servicing protected endpoints.

Zero Trust

Core principle:

> Never trust a request merely because it originated inside the network.

Authenticate and authorize every significant interaction.

mTLS

Both communicating workloads authenticate using certificates.

Service Identity

Each workload receives a machine identity rather than sharing credentials.

Secret Management

Avoid:

```
password: MyProductionPassword

```

Use platforms such as:

```
AWS Secrets Manager
Kubernetes Secrets
External Secrets

```

Token Propagation

When appropriate, carry validated user/security context across service boundaries while avoiding unnecessary exposure or excessive token scope.

---

11. Infrastructure Patterns

The document contains Sidecar, Ambassador, Adapter, Service Mesh, Anti-Corruption Layer, External Configuration Store and Distributed Configuration. [List od Mi...iples.docx | Word]

Sidecar

```
Kubernetes Pod
+-------------------+
| Spring Boot App   |
| Sidecar Container |
+-------------------+

```

Sidecar handles supporting capabilities separately from business code.

Ambassador

Proxy handles outbound interaction:

```
Application
   |
Ambassador Proxy
   |
External Service

```

Adapter

Transforms incompatible interfaces/protocols.

Example:

```
Modern REST Service
      ↓
Adapter
      ↓
Legacy SOAP API

```

Service Mesh

Infrastructure layer supports:

```
mTLS
Traffic control
Retries
Telemetry
Service-to-service security

```

Examples include Istio and Linkerd.

Anti-Corruption Layer

DDD pattern protecting your modern domain model from legacy/external models.

```
Order Domain
     ↓
ACL
     ↓
Legacy ERP model

```

Excellent during legacy modernization.

External Configuration

Keep environment configuration outside the application binary:

```
Application
     ↓
ConfigMap / Parameter Store
     ↓
Configuration

```

---

12. Cloud-Native & Kubernetes Patterns

The attached document includes Sidecar-less Service Mesh, Serverless, Operator, GitOps, Workload Identity, Multi-Cluster Mesh, Stateful Microservices, Edge Microservices, Cell-Based Architecture, Platform Engineering and IDP. [List od Mi...iples.docx | Word]

Operator Pattern

Kubernetes controller automates lifecycle management of complex applications.

```
Custom Resource
     ↓
Operator
     ↓
Create / Configure / Heal / Upgrade

```

GitOps

Git represents desired deployment state.

```
Git
 ↓
Argo CD / Flux
 ↓
Kubernetes
```

```
6
```

Infrastructure/deployment changes become version-controlled and auditable.

Workload Identity

Allow Kubernetes workloads to obtain cloud identities without long-lived AWS credentials stored inside containers.

Multi-Cluster Architecture

Use multiple Kubernetes clusters for:

- Geographic distribution
- Disaster recovery
- Regulatory isolation
- Blast-radius reduction

Cell-Based Architecture

Divide a very large platform into independent cells:

```
Users A-M -> Cell 1
Users N-Z -> Cell 2

```

Each cell contains much of the required service stack.

Benefit: Failure in one cell does not necessarily affect the entire e-commerce platform.

Platform Engineering / IDP

Provide developers a standardized internal platform:

```
Developer
   ↓
IDP / Golden Path
   ↓
Build
Test
Docker
Security Scan
Deploy
Observe

```

Useful for large organizations running tens or hundreds of microservices.

---

13. Workflow & Orchestration Patterns

The document covers Orchestration, Choreography, Workflow Engine, Process Manager and Long-Running Transactions. [List od Mi...iples.docx | Word]

Orchestration

Central component explicitly directs workflow.

Choreography

Services independently react to events.

Workflow Engine

Useful for complex workflows requiring:

- Timers
- Retry
- State persistence
- Compensation
- Human intervention

Checkout example:

```
CREATE ORDER
    ↓
RESERVE INVENTORY
    ↓
AUTHORIZE PAYMENT
    ↓
CREATE SHIPMENT
    ↓
CONFIRM ORDER

```

Process Manager

Maintains state of a multi-service business process and determines the next action.

Long-Running Transaction

A checkout/fulfilment process may continue for seconds, minutes, hours or days.

Do not keep a traditional database transaction open.

Use:

```
Saga
Events
Persistent workflow state
Compensation

```

---

14. Caching Patterns

The document contains Cache-Aside, Read-Through, Write-Through, Write-Behind and Distributed Cache. [List od Mi...iples.docx | Word]

Pattern	Meaning	E-Commerce Example
Cache-Aside	Application explicitly loads cache	Product catalog

Read-Through	Cache obtains missing data	Product lookup
Write-Through	Cache + DB updated synchronously	Customer preference
Write-Behind	Cache writes DB asynchronously	High-write non-critical use cases
Distributed Cache	Shared cache across replicas	Cart/session/product data

Example:

```
Spring Boot
    ↓
Redis
    ↓ cache miss
PostgreSQL

```

Critical consideration: Cache invalidation and stale-data tolerance must be explicitly designed.

---

15. Emerging / Modern Distributed-System Patterns

The final category in the document includes AI Gateway, Event Streaming, Data Mesh Integration, AIOps, Edge AI and Agent-Based Service patterns. [List od Mi...iples.docx | Word]

Event Streaming Architecture

Continuous event flow:

```
Orders ----\
Payments --- > Kafka -> Analytics
Inventory --/          -> Fraud Detection
                       -> Search
                       -> Data Platform

```

Different from simple queue processing because streams may be **retained, replayed and processed by multiple consumer groups**.

AI Gateway

Central controlled entry point to AI services/models supporting concerns such as authorization, model routing, usage controls and governance.

Example:

```
E-Commerce Services
       ↓
```

```
AI Gateway
       ↓
Recommendation / Search / LLM services

```

Data Mesh Integration

Business domains own analytical data products.

Example:

```
Ordering -> Order Data Product
Catalog  -> Product Data Product
Customer -> Customer Data Product

```

AIOps

Apply analytics/ML to operational telemetry for anomaly identification and operational assistance.

Edge AI Microservices

Run specialized inference closer to where low-latency processing is required.

Agent-Based Services

Autonomous AI-enabled components can coordinate specialized tasks, but business-critical actions should remain constrained by authentication, authorization, validation and audit controls.

Enterprise Principles Behind All These Patterns

Rather than memorizing the entire list independently, remember these **10 principles**:

```
1. High Cohesion
2. Loose Coupling
3. Business-Capability Ownership
4. Database/Data Ownership
5. Independent Deployment
6. Failure Isolation
7. Eventual Consistency
8. Automation
9. Observability
10. Security by Design

```

High Cohesion + Loose Coupling

Inside service:

```
Order functionality -> strongly related

```

Between services:

```
Order <-- API/Event --> Payment

```

Keep dependencies minimal.

Independent Deployability

You should be able to release:

```
payment-service:v2.7.1

```

without redeploying Catalog, Inventory and Order services.

Design for Failure

Always assume:

```
Network can fail.
Pod can restart.
Database can become unavailable.
Message can arrive twice.
Request can time out.
Dependency can become slow.

```

Therefore design for:

```
Timeout + Retry + Circuit Breaker
Idempotency
DLQ
Observability
Compensation

```

Most Important Patterns for Your Java/Spring Boot Interview

The attached document highlights API Gateway, BFF, Database per Service, Service Discovery, Circuit Breaker, Saga, CQRS, Event Sourcing, Event-Driven Architecture, Service Mesh, Sidecar, Distributed Tracing, Transactional Outbox, Strangler Fig and DDD as architecturally important patterns. [List od Mi...iples.docx | Word]

For an **enterprise Java/Spring Boot e-commerce interview**, I would prioritize them in this practical order:

Tier 1: Must Know

```
DDD / Bounded Context
        ↓
Database per Service
        ↓
REST + API Gateway
        ↓
Event-Driven Architecture
        ↓
Saga
        ↓
Transactional Outbox
        ↓
Circuit Breaker / Retry / Timeout
        ↓
Service Discovery
        ↓
Docker + Kubernetes
        ↓
Observability
```

```
20
```

Tier 2: Senior Developer / Lead

```
BFF
CQRS
Cache-Aside
DLQ
Strangler Fig
Idempotency
Distributed Tracing
Compensating Transactions
Anti-Corruption Layer
Canary / Blue-Green
```

Tier 3: Architect

```
Event Sourcing
Service Mesh
GitOps
Cell-Based Architecture
Platform Engineering
Multi-Cluster
Workload Identity
Workflow Engines
Polyglot Persistence
Event-Carried State Transfer
```

One Complete E-Commerce Order Flow

This is the best way to connect the patterns rather than memorize them separately:

```
Customer
   |
   v
API Gateway
   |
   v
Order Service
   |
   +-- JPA/Hibernate --> Order Database
   |
   +-- Transactional Outbox
               |
               v
            Kafka
               |
      OrderCreated Event
        /      |       \
       v       v        v
 Inventory   Payment  Notification
 Service     Service     Service
    |           |
Reserve       Authorize
Stock         Payment
    |           |
    +---- Saga -+
          |
      Success
          |
    OrderConfirmed
```

Cross-cutting around it:

```
Resilience
  -> Timeout
  -> Retry
  -> Circuit Breaker
  -> DLQ

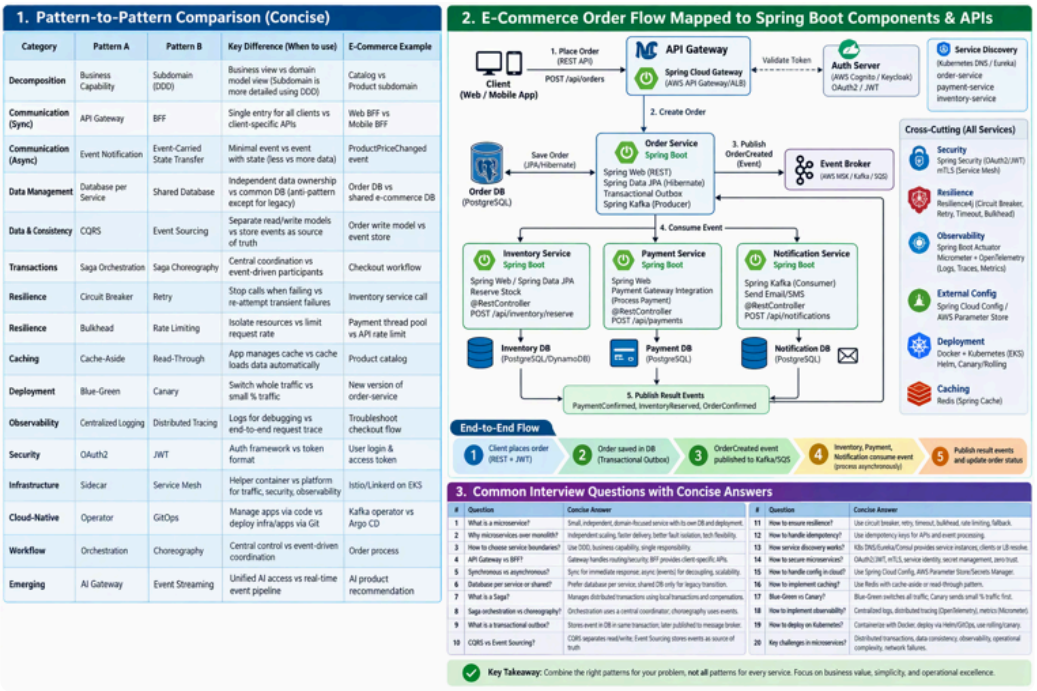
Security
  -> OAuth2
  -> JWT
  -> mTLS
  -> Secrets

Observability
  -> OpenTelemetry
  -> Trace ID
  -> Metrics
  -> Central Logs

Deployment
  -> Docker
  -> Kubernetes/EKS
  -> Rolling/Canary
  -> GitOps

```

That is essentially the **enterprise-level picture** you should be able to explain in an interview: **DDD determines the service boundaries; Database-per-Service establishes data ownership; REST handles immediate interactions; events decouple asynchronous workflows; Saga manages cross-service business consistency; Outbox reliably publishes state changes; resilience patterns contain failures; OAuth/JWT/workload identity secure calls; OpenTelemetry makes distributed requests observable; and Docker/Kubernetes/AWS provide independently deployable and scalable runtime infrastructure.**



1. Concise Pattern-to-Pattern Comparison

Pattern A	Pattern B	Key Difference	E-Commerce Use
Business Capability	DDD Subdomain	Business function vs detailed domain boundary	Catalog service vs Product bounded context
API Gateway	BFF	Common entry point vs client-specific backend	Gateway routing vs Mobile/Web BFF
REST	Event-Driven	Immediate response vs asynchronous processing	Product lookup vs OrderPlaced event
API Composition	Aggregator	Composition technique vs dedicated coordinating component	Combined order summary
Event Notification	Event-Carried State Transfer	Event identifier only vs event containing relevant state	Product ID vs product ID with new price
Database per Service	Shared Database	Independent ownership vs schema coupling	Separate Order and Payment databases
CQRS	Event Sourcing	Separate reads/writes vs store state as events	Order read model vs order event history
Saga Orchestration	Saga Choreography	Central coordinator vs distributed event reactions	Checkout workflow
2PC	Saga	Strong distributed	Prefer Saga for cloud

		commit vs eventual consistency with compensation	microservices
Circuit Breaker	Retry	Stop calls during failure vs repeat transient calls	Inventory API protection
Timeout	Circuit Breaker	Limits call duration vs blocks unhealthy dependency	Payment API call
Bulkhead	Rate Limiting	Isolates internal resources vs controls request volume	Payment pool vs checkout API limit
Cache-Aside	Read-Through	Application loads cache vs cache provider loads data	Product catalog caching
Blue-Green	Canary	Switch full traffic vs test with small traffic percentage	New Order Service release
Centralized Logging	Distributed Tracing	Aggregated logs vs end-to-end request path	Checkout failure investigation
OAuth2	JWT	Authorization framework vs token format	Login authorization and access token
Sidecar	Service Mesh	Per-Pod helper vs infrastructure-wide communication layer	Proxy sidecar vs Istio/Linkerd

Orchestration	Choreography	Central control vs decentralized events	Order fulfilment
Operator	GitOps	Automates resource lifecycle vs reconciles Git state	Kafka Operator vs Argo CD

3. Common Interview Questions with Concise Answers

1. What is a microservice?

A small, independently deployable service aligned with one business capability and normally owning its data.

2. How do you identify microservice boundaries?

Use **DDD bounded contexts, business capabilities, high cohesion and single responsibility**. Avoid splitting services by technical layers.

3. Why use microservices instead of a monolith?

For independent deployment, selective scaling, fault isolation, team autonomy and faster delivery. A monolith is often simpler for small systems.

4. What is Database per Service?

Each service owns its database or schema. Other services access the data through APIs or events, not direct table queries.

5. Why should services not share database tables?

Shared tables create schema coupling, bypass business rules and prevent independent deployment.

6. When should REST communication be used?

When an immediate response is required, such as checking product availability before displaying a checkout page.

7. When should asynchronous messaging be used?

For decoupled workflows, long-running processing, traffic buffering and operations that do not require an immediate response.

8. API Gateway vs BFF?

An **API Gateway** provides common routing, security and rate limiting. A **BFF** exposes APIs optimized for a specific client, such as mobile or web.

9. What is a Saga?

A distributed business transaction implemented through local service transactions and compensating operations.

10. Saga orchestration vs choreography?

Orchestration uses a central coordinator. Choreography allows services to react to events without a central controller.

11. What is a compensating transaction?

A business action that reverses an earlier completed step, such as refunding payment and releasing inventory after order failure.

12. What is the Transactional Outbox Pattern?

The service saves business data and an outbox record in one database transaction. A separate publisher reliably sends the event to Kafka or AWS messaging.

13. What problem does CQRS solve?

CQRS separates command and query models, allowing read and write workloads to evolve and scale independently.

14. CQRS vs Event Sourcing?

CQRS separates reads and writes. Event Sourcing stores domain events as the source of truth. They can be used independently.

15. How do you handle duplicate messages?

Use idempotent consumers, event IDs, unique database constraints and a processed-event table.

16. What is eventual consistency?

Different services may temporarily show different states, but events eventually bring all affected services to a consistent business state.

17. How do you make REST calls resilient?

Configure strict timeouts, bounded retries, circuit breakers, bulkheads and appropriate fallbacks using Resilience4j.

18. When should a request be retried?

Only for transient failures and preferably for idempotent operations. Never blindly retry payment capture or order creation.

19. What is a Circuit Breaker?

It temporarily blocks calls to an unhealthy dependency, preventing repeated failures and resource exhaustion.

20. What is the Bulkhead Pattern?

It isolates thread pools, connections or concurrency so one failing dependency cannot exhaust the whole service.

21. How does service discovery work in Kubernetes?

Services communicate through stable Kubernetes Service names and cluster DNS while Kubernetes routes traffic to healthy Pods.

22. Is Eureka required on Kubernetes?

Usually not. Kubernetes Services and DNS already provide native service discovery. Eureka may remain useful in non-Kubernetes or hybrid environments.

23. How do you secure Spring Boot microservices?

Use Spring Security, OAuth2/OIDC, JWT validation, authorization scopes, workload identities, mTLS and managed secrets.

24. How do you implement observability?

Use structured centralized logs, correlation IDs, Micrometer metrics, OpenTelemetry tracing and Spring Boot Actuator health endpoints.

25. Liveness vs readiness probe?

Liveness determines whether Kubernetes should restart a container. Readiness determines whether the Pod should receive traffic.

26. How do you cache product data?

Use Spring Cache with Redis and the Cache-Aside Pattern. Define TTL, eviction and invalidation rules explicitly.

27. How do you deploy without downtime?

Use Kubernetes rolling updates, readiness probes and backward-compatible APIs and database migrations. Use canary deployment for higher-risk releases.

28. How do you migrate a monolith to microservices?

Apply the Strangler Fig Pattern by extracting one bounded context at a time and routing traffic through an API Gateway.

29. What are the main microservice challenges?

Distributed transactions, eventual consistency, network failures, observability, security, deployment automation and increased operational complexity.

30. Explain the complete order flow.

The Gateway validates the JWT and routes `POST /api/orders` to Order Service. Order Service saves the order and outbox event using JPA/Hibernate. The event is published to Kafka or AWS messaging. Inventory, Payment and Notification services process it, publish result events and allow the Saga to confirm or compensate the order.